

Firmware de un decodificador MPEG-2 implementado en FPGA

1st Esteban Adriel Bustamante
Universidad Tecnológica Nacional
Facultad Regional La Rioja
La Rioja, Argentina
ebustamante896@gmail.com

Resumen—This work aims to implement the firmware (specifically for this case 'the driver') for a MPEG-2 decoder implemented in a Polarfire®MPFS095T FPGA. This implies porting the design initially designed for Xilinx ML505 development board, and then designing and implementing the OS and the actual tasks that will altogether constitute the firmware as a whole.

Index Terms—firmware, FPGA, SoC, software, Verilog.

I. INTRODUCCIÓN

El objetivo de este proyecto es la implementación del firmware de un bloque de hardware implementado en FPGA. Para nuestro caso este bloque de hardware es un decoder MPEG-2 conforme a la norma En primer lugar se muestra en la figura 3 el sistema de microprocesadores con el que se va a trabajar. Estamos hablando de la placa PolarFire SoC FPGA Discovery Kit, que además de contener los procesadores mencionados, tiene una FPGA MPFS095T.

II. METODOLOGÍA

En el diseño típico de un sistema de punto fijo se emplea el flujo observado en la Figura 1. Si bien el sistema ya se encuentra implementado, corresponderá al proyecto actual implementarlo en el SoC PolarFire MPFS095T, corriendo las diferentes pruebas ofrecidas por el diseñador del sistema para verificarlo a nivel comportamental y funcional. Cabe destacar que este algoritmo está totalmente implementado en hardware.

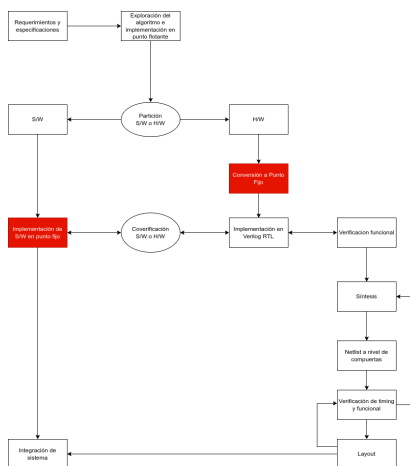


Figura 1. Flujo típico de diseño de sistema en punto fijo

Para la implementación del firmware se utiliza la metodología TDD (*Test Driven Development*), que como lo indican sus siglas se basa en el desarrollo guiado por pruebas, y se encuentra ampliamente difundido en la actualidad en el mundo del software embebido. En la Figura 2 observamos el ciclo de desarrollo típico de esta metodología. Son numerosas las ventajas de esta decisión de diseño, pero la principal es que al ser el primordial objetivo conseguir un firmware de nivel industrial que se destaque por su robustez y flexibilidad, es esencial conseguir un nivel de *coverage* del 100 %.

El framework de TDD elegido para desarrollar el código es *Ceedling*, pensado para desarrollar firmware para embebidos en ANSI C, lo cual es parte del proyecto. Para la administración del proyecto se hace uso de la herramienta *Org Mode* + *Projectile*, que permitirá controlar el cumplimiento de los plazos establecidos en la planificación.

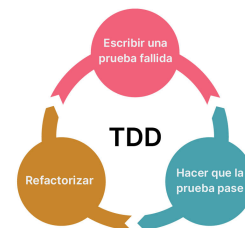


Figura 2. Desarrollo guiado por pruebas

En cuanto a la documentación, decir que se realiza con las herramientas Doxygen en conjunto con las *Github Pages*, que permiten automatizar el flujo de publicación de la misma.

Por ultimo recalcar que para la programación de la placa de desarrollo, es decir del FPGA se hace uso del software provisto por el fabricante Libero®, y para la programación del sistema de microprocesadores se hará uso de otro software del fabricante llamado Softconsole®. En la figura 3 se muestra el flujo completo para llegar a un nivel de producción del firmware.

III. CODEC MPEG-2

Se presenta ahora el diseño en hardware en el que basamos nuestro trabajo. Se trata de un diseño *open-source*, (ver Ref. [1]) implementado en Verilog.

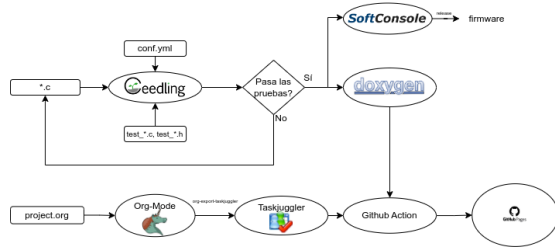


Figura 3. Flujo de trabajo de software completo

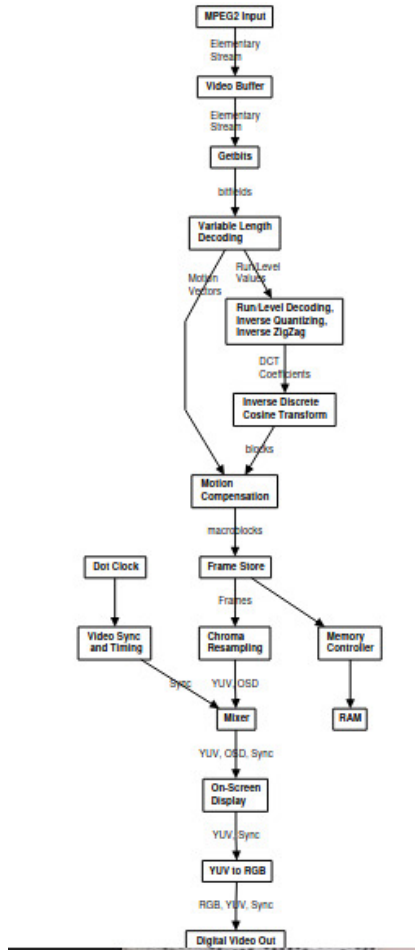


Figura 4. Pipeline del Decodificador MPEG-2

En la Figura 4 se muestra el diagrama en bloques completo del diseño. Ahora describimos brevemente cada una de sus partes:

1. **MPEG-2 Input:** Recibe la secuencia comprimida en formato MPEG-2, que puede ser un flujo de transporte o un flujo elemental de video.
2. **Elementary Stream Strip:** Extrae únicamente el flujo elemental de video del flujo de entrada.
3. **Video Buffer:** Almacena temporalmente el flujo de bits para asegurar una alimentación continua al decodificador.
4. **Syntax Parser:** Analiza la sintaxis del flujo de bits

para identificar cabeceras, parámetros y estructuras de control.

5. **Variable Length Decoding (VLD):** Decodifica los símbolos comprimidos con codificación de longitud variable (como Huffman), produciendo coeficientes y vectores de movimiento.
6. **Run-Level Decoding, Inverse Quantizing, Inverse Zigzag:**
 - *Run-Level Decoding:* Reconstruye la secuencia de coeficientes.
 - *Inverse Quantizing:* Restaura la escala original de los valores cuantificados.
 - *Inverse Zigzag:* Reordena los coeficientes a su formato 2D original.
7. **Inverse Discrete Cosine Transform (IDCT):** Convierte los coeficientes del dominio de la frecuencia al dominio espacial, generando datos de píxeles.
8. **Motion Compensation:** Usa cuadros anteriores o futuros y vectores de movimiento para reconstruir bloques predictivos en cuadros P y B.
9. **Frame Store:** Almacena los cuadros decodificados que servirán como referencia para la predicción por movimiento.
10. **Chroma Resampling:** Convierte el muestreo de color de formatos como 4:2:0 a formatos como 4:2:2 o 4:4:4 para mejorar la calidad visual. En nuestro caso se usa siempre el muestreo 4:2:0.
11. **Memory Controller / RAM:** Gestiona el acceso a la memoria externa para almacenar cuadros, referencias y buffers intermedios.
12. **Video Sync and Timing:** Genera las señales de sincronización necesarias para mostrar correctamente los cuadros en la pantalla.
13. **Mixer:** Mezcla el video con elementos gráficos superpuestos como menús o subtítulos (OSD).
14. **On-Screen Display (OSD):** Agrega elementos gráficos o informativos sobre la imagen de video.
15. **YUV to RGB:** Convierte el video del espacio de color YUV al espacio RGB, usado por la mayoría de las pantallas.
16. **Digital Video Out:** Produce la salida final del video ya decodificado y preparado para su visualización.
17. **Dot Clock:** Proporciona la señal de reloj que controla el ritmo de presentación de los píxeles en pantalla.

IV. SOC DONDE SE IMPLEMENTARÁ EL PROYECTO

El SoC fue desarrollado por la empresa Microchip®, y se trata de un MPFS095T, embebido a su vez en la placa de desarrollo *Discovery@Kit*, que contiene una diversidad de periféricos y del programador del SoC. En las figuras 5 y 6 incluimos figuras del kit y del subsistema de microprocesadores que se encuentra en el mismo. Esta última información nos resulta particularmente útil para el desarrollo del firmware.

Para poder portar nuestro diseño es necesario tener en cuenta algunos detalles:

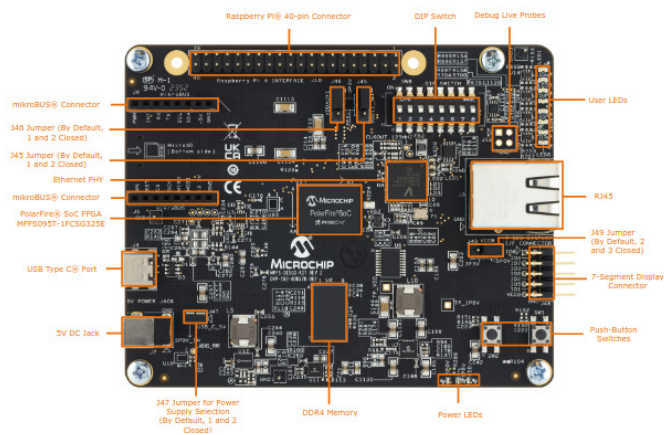


Figura 5. Discovery® Kit

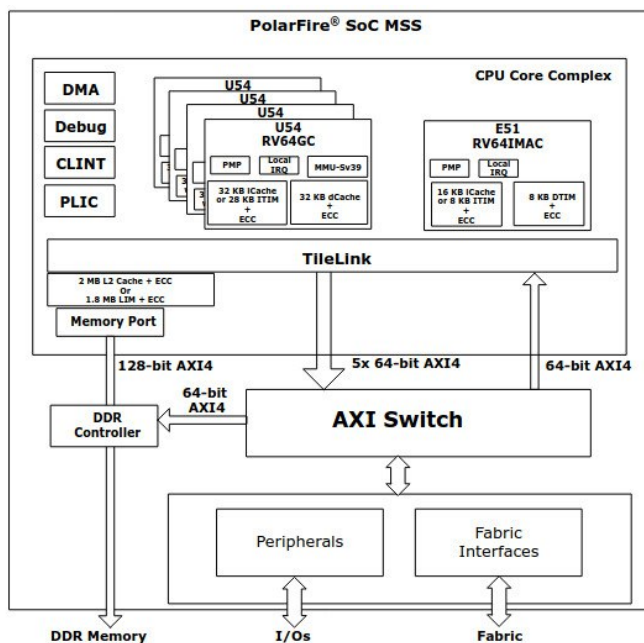


Figura 6. Diagrama en bloques del MSS(Microprocessor Sub-System)

REFERENCIAS

- [1] K. De Vleeschauwer, "MPEG-2 video decoder," Agosto 2017. [Online]. Available: "https://opencores.org/projects/mpeg2fpga"
- [2] J. Watkinson, *The MPEG handbook: MPEG-1, MPEG-2, MPEG-4*. Focal Press, Taylor & Francis Group, 2013.
- [3] R. Cox, F. Kaashoek, and R. Morris, *xv6, un sistema operativo de enseñanza desarrollado en el MIT*. Massachusetts Institute of Technology, 2009, disponible en: https://pdos.csail.mit.edu/6.828/2018/xv6/.
- [4] P. Pessolani, "Minix4rt: A real-time operating system based on minix," Master's thesis, Universidad Nacional de La Plata, 2006.
- [5] C. Billaud, "Mise en oeuvre d'un décodeur MPEG-2 sur architecture mixte processeur + FPGA," Master's thesis, ENSEA, 2004.
- [6] I. O. for Standardization, "Information technology — MPEG video technologies — Part 7: Versatile supplemental enhancement information messages for coded video bitstreams," International Organization for Standardization, Geneva, CH, Standard, Mar. 2024.

ÍNDICE DE FIGURAS

1.	Flujo típico de diseño de sistema en punto fijo .	1
2.	Desarrollo guiado por pruebas	1
3.	Flujo de trabajo de software completo	2
4.	Pipeline del Decodificador MPEG-2	2
5.	Discovery®Kit	3
6.	Diagrama en bloques del MSS(Microprocessor Sub-System)	3

- En primer lugar, el diseño original incluía el uso de un DDR2, mientras que este hace uso de un DDR4 ya disponible en la placa por lo que prácticamente el controlador del mismo debe ser reescrito por completo.
- Por otro lado en el diseño original se incluye un pequeño stack TCP/IP para poder interactuar a través de Ethernet con la placa. A partir de esta pequeña implementación podría generarse un controlador completo de Ethernet, teniendo en cuenta las altas capacidades en cuanto a cantidad de elementos lógicos para la misma.
- Por último también deberán ser escritos los constraints correspondientes para el nuevo hardware sobre el que se va a implementar el diseño.